

Day 26: Logistic Regression (Binary Classification Engine)

CORE THEORY, DERIVATIONS, AND LOSS FUNCTIONS

1. Core Theory: Logistic Regression & Its Necessity

Until now, predictions in Linear Regression yielded continuous target outputs such as Income, Price, or Sales.

Abhi tak, Linear Regression mein jo bhi aapne predict kiya tha wo ek continuous number tha jaise Income, Price, ya Sales.

But in real life, many times we don't need a continuous number, rather we need to predict Yes/No, Pass/Fail, True/False, or 0 / 1 (Binary Classification).

Lekin real life mein, bahut baar hamein continuous number ki zaroorat nahi hoti, balki hamein Yes/No, Pass/Fail, True/False, ya 0/1 predict karna hota hai (Binary Classification).

Examples of Binary Classification:

- **Retail Optimization:** Predicting whether a customer will purchase high-value jewelry or not (Yes/No).

Will this customer buy jewelry or not? (Yes/No)

- **Risk Assessment:** Determining if a credit card transaction is fraudulent or legitimate (Fraud/Safe).

Is this credit card transaction fraud? (Fraud/Safe)

Why Linear Regression Fails for Classification:

Applying a continuous linear equation $Y = mX + c$ to predict bounded discrete classes introduces two fundamental flaws:

Agar hum Yes/No (1 ya 0) predict karne ke liye ek continuous straight line $Y = mX + c$ apply karte hain, toh do major problems aati hain:

1. **Out of Bounds Predictions:** A linear function can generate values scaling to positive or negative infinity. It produces uninterpretable predictions like 1.5 or -0.5, whereas absolute classification probability must strictly lie within the interval $[0, 1]$.

Out of Bounds: Straight line infinity tak chali jati hai. Yeh 1.5 ya -0.5 jaise predictions de sakti hai. Lekin probability strictly 0 aur 1 ke beech honi chahiye!

2. **Extreme Outliers Sensitivity:** The presence of extreme outliers shifts the ordinary least squares regression line significantly, leading to heavily distorted classification thresholds and erroneous predictions for existing data points.

Extreme Outliers: Agar data mein koi bahut bada outlier aa jaye, toh linear regression line poori tarah tilt ho jati hai, jisse wrong predictions milti hain.

2. Logistic Regression's Core Solution: The Sigmoid Function

Logistic regression builds upon the linear base layer $mX + c$ but introduces an architectural constraint called the Sigmoid Function (or Logistic Function).

Logistic Regression bhi $mX + c$ se shuru hota hai, lekin iske bahar ek guard laga deta hai jise Sigmoid Function (ya Logistic Function) kehte hain.

The Sigmoid transformation compresses any arbitrary real-valued input across positive or negative infinity into a strict probability interval between 0 and 1.

Sigmoid ek mathematical capsule hai jo kisi bhi positive ya negative infinity number ko leta hai aur usko 0 aur 1 ke beech ek precise value mein squeeze kar deta hai.

Human-Readable Sigmoid Formula:

Padhne me easy formula:

$$P = 1 / (1 + e^{-Z})$$

Where $Z = mX + c$ and e is Euler's constant (approximately 2.718).

Yahan, $Z = mX + c$ aur e Euler's constant hai (approx 2.718).

- As Z approaches a very large positive number, the Probability moves close to 1.
Agar Z ek bahut bada positive number hai, toh Probability 1 ke paas chali jati hai.
- As Z approaches a very large negative number, the Probability moves close to 0.
Agar Z ek bahut bada negative number hai, toh Probability 0 ke paas chali jati hai.
- When $Z = 0$, the probability yields exactly the midpoint value of 0.5 (50%).
Agar $Z = 0$ hai, toh Probability ekdum beech mein hogi, jo ki 0.5 (50%) hai.

3. Decision Boundary

Once the model computes the continuous probability score via the Sigmoid transformation, a thresholding function maps the decimal probability to a final discrete category. The baseline standard decision boundary is set at 0.5:

Ek baar jab computer Sigmoid se probability calculate kar leta hai (jaise 0.75), toh hum final decision lene ke liye ek threshold apply karte hain, jise Decision Boundary kehte hain. Standard boundary 0.5 hai:

If Probability $\geq 0.5 \rightarrow$ Output = 1 (Yes / Success)
If Probability $< 0.5 \rightarrow$ Output = 0 (No / Failure)

4. Historical Context & Mathematical Evolution

Origins and Target Application (1838–1844)

The mathematical foundation of this function was discovered by Belgian mathematician Pierre Franois Verhulst during his investigations into population dynamics and epidemiological modeling.

Yeh formula sabse pehle ek Belgian mathematician Pierre Franois Verhulst ne 1838 aur 1844 ke saal ke beech discover kiya tha.

His Target: He was studying the pattern of a disease spreading or a country's Population Growth.

Unka Target: Wo kisi bimari ke phailne ke pattern ya kisi desh ki Population Growth ko study kar rahe the.

The Problem: If we use the old straight line $Y = mX + c$ for population growth, linear math says population will keep growing forever. But this doesn't happen in real life!

Problem: Agar hum population growth ke liye purani straight line $Y = mX + c$ use karein, toh linear math kehta hai ki population hamesha badhti rahegi. Lekin real life mein aisa nahi hota!

The Reality: Resources, food, and land are limited in a country (called Carrying Capacity). Population grows rapidly at first, but freezes flat after a limit when resources run out. Verhulst needed a formula fixed at 0 at the bottom and stopping at a maximum ceiling at the top. In this research, he created the Logistic Function!

Reality: Kisi desh mein resources, khana, aur zameen limited hote hain (jise Carrying Capacity kehte hain). Population pehle tezi se badhti hai, lekin resources khatam hone par ek limit ke baad freeze ho jati hai. Verhulst ko ek aisa formula chahiye tha jo neeche (start) 0 par fixed ho aur upar ek maximum ceiling (limit) par ruk jaye. Is research mein unhone Logistic Function banaya!

Mathematical Derivation from Odds to Probability

Verhulst did not just write down the direct formula. He derived its relationship from Odds (the chance of winning versus losing). Let's see the math from scratch:

Verhulst ne sirf direct formula nahi likha. Unhone iska relationship Odds (jeetne aur haarne ka chance) se derive kiya. Aaiye shuru se math dekhte hain:

Step A: Probability vs. Odds

Probability (P) means the chance of an event happening (which is between 0 and 1). But in betting or gaming, we look at Odds:

Probability (P) ka matlab hai kisi event ke hone ka chance (jo 0 aur 1 ke beech hota hai). Lekin betting ya gaming mein hum Odds dekhte hain:

$$\text{Odds} = P / (1 - P)$$

Example: If Probability is 0.8, the chance of not happening is 0.2. $\text{Odds} = 0.8 / 0.2 = 4$ (Meaning the chance of happening is 4 times higher than not happening). The range of Odds can go from 0 to infinity.

Example: Agar Probability 0.8 hai, toh na hone ka chance 0.2 hai. $\text{Odds} = 0.8 / 0.2 = 4$ (Yaani hone ka chance na hone se 4 guna zyada hai). Odds ki range 0 se infinity tak ja sakti hai.

Step B: The Logarithmic Transformation (Log-Odds / Logit)

Mathematicians noticed that Odds go from 0 to infinity, but they needed a number that goes from Minus Infinity to Plus Infinity, so it matches our $mX + c$. They took the log (Natural Log - base e) of the Odds. This is called the Logit Function:

Mathematicians ne notice kiya ki Odds 0 se infinity tak jate hain, lekin unhein ek aisa number chahiye tha jo Minus Infinity se Plus Infinity tak jaye, taaki wo hamare $mX + c$ se match kare. Unhone Odds ka log (Natural Log - base e) liya. Isse Logit Function kehte hain:

$$\text{Log-Odds} = \log_e (P / (1 - P))$$

Now, the beauty of this Log-Odds is that its range became exactly like a straight line: Minus Infinity to Plus Infinity. Now we can set it equal to our old linear equation:

Ab, is Log-Odds ki beauty yeh hai ki iski range ekdum straight line jaisi ho gayi: Minus Infinity se Plus Infinity. Ab hum ise apni purani linear equation ke barabar rakh sakte hain:

$$\log_e (P / (1 - P)) = mX + c = Z$$

Step C: Algebraic Extraction of Probability (P)

He inverted this equation to get the precise formula for Probability (P):

Unhone Probability (P) ka precise formula nikalne ke liye is equation ko invert kiya:

1. Removing log by applying exponential (e) on both sides: $P / (1 - P) = e^Z$

Both sides par exponential (e) apply karke log hatana.

2. Cross multiplication: $P = e^Z \times (1 - P) \Rightarrow P = e^Z - P \cdot e^Z$

Cross multiplication karna.

3. Bringing all P terms to the left side: $P + P \cdot e^Z = e^Z$

Saare P wale terms ko left side lana.

4. Taking P as a common factor: $P \cdot (1 + e^Z) = e^Z$

P ko common factor lena.

5. Shifting $(1 + e^Z)$ to the denominator to isolate P: $P = e^Z / (1 + e^Z)$

$(1 + e^Z)$ ko denominator mein bhejkar P ko akela karna.

6. Standard Form (Dividing top and bottom by e^Z): $P = 1 / (1 + e^{-Z})$

Standard Form (Upar aur neeche ko e^Z se divide karna).

How Does It Stay Locked Between 0 and 1?

No matter what infinity number you put into this capsule from the world, it behaves like this:

Duniya ka koi bhi infinity number aap is capsule mein daalein, yeh aise hi behave karta hai:

- **Case 1: Z is a very large positive number ($+\infty$)**

If $Z = \infty$ then $e^{-\infty} = 0$. $P = 1 / (1 + 0) = 1$ [Max Ceiling Lock]

Agar Z ek bahut bada positive number hai, toh probability 1 par lock ho jati hai.

- **Case 2: Z is a very large negative number ($-\infty$)**

If $Z = -\infty$, then $e^{-(-\infty)} = e^{\infty} = \infty$. $P = 1 / (1 + \infty) = 0$ [Min Floor Lock]

Agar Z ek bahut bada negative number hai, toh probability 0 par lock ho jati hai.

- **Case 3: Z is exactly in the middle (0)**

If $Z = 0$, then $e^{-0} = 1$. $P = 1 / (1 + 1) = 0.5$ [50% Mid Boundary]

Agar Z ekdum beech mein (0) hai, toh probability exactly 0.5 hoti hai.

5. Cost Function: Log Loss (Binary Cross-Entropy)

The Betting Logic (Coin Toss & Probability)

Suppose you toss a coin. The probability of getting a Head is P , so the probability of a Tail is $(1 - P)$. Basic math rule states that if two independent events happen together, we Multiply them to find the Combined Probability:

Maan lijiye aap ek coin toss karte hain. Head aane ki probability P hai, toh Tail aane ki $(1 - P)$ hogi. Basic math rule kehta hai ki agar do independent events sath mein hote hain, toh hum unhein Multiply karte hain Combined Probability nikalne ke liye:

$$\text{Total Probability} = P \times (1 - P)$$

Pierre-Simon Laplace's Masterstroke:

Mathematicians created a single master power equation that works as an automatic switch without needing conditions:

Mathematicians ne ek single master power equation banayi jo bina kisi condition ke ek automatic switch ki tarah kaam karti hai:

$$\text{Combined Probability} = P^Y \times (1 - P)^{(1 - Y)}$$

- Case A (If $Y = 1$): The formula becomes $P^1 \times (1 - P)^0 = P \times 1 = P$

Agar $Y = 1$ hai, toh formula sirf P reh jata hai.

- Case B (If $Y = 0$): The formula becomes $P^0 \times (1 - P)^1 = 1 \times (1 - P) = (1 - P)$

Agar $Y = 0$ hai, toh formula sirf $(1 - P)$ reh jata hai.

The Entry of Logarithm

The computer needed to find the differentiation (gradient) of this dynamic function, but calculating derivatives of powers is highly complex. So, they took the log of this entire formula. The log rule states: $\log(A \times B) = \log(A) + \log(B)$ and $\log(A^B) = B \times \log(A)$.

Computer ko is dynamic function ka differentiation (gradient) nikalna tha, lekin powers ka derivative nikalna bahut complex hota hai. Isliye, unhone is poore formula ka log le liya. Log rule kehta hai ki guna plus mein badal jata hai aur power aage aa jati hai.

$$\log [P^Y \times (1 - P)^{(1 - Y)}] = [Y \times \log(P)] + [(1 - Y) \times \log(1 - P)]$$

The Nature of Logarithm (The Negative Nature)

The Sigmoid function always outputs a probability (P) that is a decimal number between 0 and 1. It is a rule of mathematics that whenever you take the log of a decimal number smaller than 1, the answer will always be minus (negative):

Sigmoid function hamesha ek probability (P) output karta hai jo 0 aur 1 ke beech ek decimal number hota hai. Yeh math ka rule hai ki jab bhi aap 1 se chhote decimal number ka log lenge, answer hamesha minus (negative) aayega:

$$\log(0.9) = -0.045 \mid \log(0.5) = -0.301 \mid \log(0.01) = -2.000$$

What Out-of-Logic Chaos Happens If We Don't Apply Minus (-)?

Agar hum bina minus sign ke error formula likhein jab customer ne actually jewelry kharidi ho ($Y = 1$): $Loss = \log(P)$

- Situation A (Model Performed Excellent): The model gave 90% probability ($P = 0.9$). **Loss = $\log(0.9) = -0.045$**
Model ne bohot achha kaam kiya aur 90% probability di.
- Situation B (Model Performed Horribly): The model gave only 1% probability ($P = 0.01$). **Loss = $\log(0.01) = -2.000$**
Model ne bohot kharab kaam kiya aur sirf 1% probability di.

In human and computer logic, a larger negative number is actually smaller in value ($-2.000 < -0.045$). This means without a minus, the computer will think a horrible prediction yielded less loss while a great prediction yielded more loss! The whole logic flips upside down.

Human aur computer logic mein, ek bada negative number actually value mein chhota hota hai ($-2.000 < -0.045$). Iska matlab bina minus ke, computer sochega ki ek ghatiya prediction mein kam loss hua aur ek achhi prediction mein zyada loss hua! Poora logic ulta ho jata hai.

Solution: The Magic of the Minus Sign (The Inverter)

Pierre François Verhulst and other statisticians resolved this negative confusion by placing a Minus (-) sign outside the brackets beforehand.

$$Loss = - [Y \times \log(P) + (1 - Y) \times \log(1 - P)]$$

Statisticians ne is minus confusion ko brackets ke baahar pehle se ek Minus (-) sign lagakar door kiya. Isse hamesha positive scale milta hai.

- On Great Predictions ($P = 0.9$): **Loss = $-\log(0.9) = -(-0.045) = +0.045$**
Achhi prediction par loss zero ke bohot paas aata hai.
- On Horrible Predictions ($P = 0.01$): **Loss = $-\log(0.01) = -(-2.000) = +2.000$**
Kharab prediction par loss value positive aur badi ho jati hai.

6. Practical Numerical Example (Step-by-Step Math Block)

Scenario: We are training a model to check: "Will the customer buy high-value gold jewelry (1) or not (0)?"

Scenario: Hum ek model ko train kar rahe hain check karne ke liye: "Kya customer high-value gold jewelry kharidega (1) ya nahi (0)?"

- X: Customer's Past Purchase Score = 2.0
- Y: Actual Output = 1 (The customer actually bought the jewelry)

- Parameters: Weights temporarily scale: $m = 1.5$, $c = -2.0$

Step 1: Calculate Linear Combination (Z)

$$Z = mX + c \Rightarrow Z = (1.5 \times 2.0) + (-2.0) = 3.0 - 2.0 = 1.0$$

Step 2: Pass through Sigmoid Function to get Probability (P)

$$P = 1 / (1 + e^{-Z}) = 1 / (1 + e^{-1.0})$$

Since $e^{-1.0} \approx 0.3678$:

$$P = 1 / (1 + 0.3678) = 1 / 1.3678 \approx 0.731 \text{ (73.1\% Probability)}$$

Step 3: Calculate Individual Row Log Loss Value

$$\text{Loss} = - [1 \times \log(0.731) + (1 - 1) \times \log(1 - 0.731)] = - [-0.313] = +0.313$$

Conclusion & Decision Boundary Check: Since $P = 0.731 \geq 0.5$, the final predicted category output is 1 (Yes). The positive error score of the model is 0.313. Gradient Descent will use this loss score to update m and c until the gradient becomes completely zero.

Conclusion: Kyunki $P = 0.731 \geq 0.5$ hai, toh final predicted category output 1 (Yes) hai. Model ka positive error score 0.313 hai. Gradient Descent is loss score ka use karke m aur c ko update karega jab tak gradient zero na ho jaye.

7. Calculate Gradients Using Predicted Probability (P)

Ab computer hamare is initial andaze ($P \approx 0.731$) aur asaliyat ($Y = 1$) ka farq nikaalega taaki m aur c ko sahi direction mein badhaya ja sake.

Now the computer will calculate the difference between our initial guess and reality to push m and c in the right direction.

Formula for Intercept Gradient (**Gradient_c**):

$$\text{Gradient}_c = (P - Y) = (0.731 - 1) = -0.269$$

Formula for Slope Gradient (**Gradient_m**):

$$\text{Gradient}_m = X \times (P - Y) = 2.0 \times (0.731 - 1) = -0.538$$

8. Update Parameters (m and c) to Pavitr State

Ab computer in dono gradients ko Gradient Descent Update Rule wale formula mein daalega taaki fake weights ko real weights ki taraf dhakela ja sake. (Maan lijiye hamara Learning Rate (η) = 0.1 hai)

Now the computer will plug these two gradients into the Gradient Descent Update Rule formula to push fake weights toward real weights. (Let's assume our Learning Rate = 0.1)

Calculating New m:

$$\text{New } m = \text{Old } m - (\eta \times \text{Gradient}_m) = 1.5 - (0.1 \times -0.538) = 1.5538$$

Calculating New c:

$$\text{New } c = \text{Old } c - (\eta \times \text{Gradient}_c) = -2.0 - (0.1 \times -0.269) = -1.9731$$

9. Optimization Loop: Loop Dobara Ghoomne Ka Niyam

Yahan tak pahunchte hi computer ka Pehla Chakkar (Epoch 1) poora hota hai. Ab purane ghatiya parameters ($m=1.5$, $c=-2.0$) clear ho jayenge aur unki jagah yeh naye, thode se behtar (pavitr) parameters ($m=1.5538$, $c=-1.9731$) le lenge.

As soon as we reach here, the computer's First Cycle (Epoch 1) is complete. Now old parameters will clear out and these new, slightly better parameters will take over.

Iske Baad Loop Dobara Kaise Ghoomega?

How will the loop run again after this?

Computer in Naye m aur c ko lekar turant bina ruke Chakkar Number 2 (Epoch 2) shuru karega. In naye numbers se wapas Step 1 chalega aur naya Z niklega:

The computer will take these new m and c and immediately start Cycle Number 2 (Epoch 2) without stopping. From these new numbers, Step 1 will run again to find the new Z:

$$Z_{\text{new}} = (1.5538 \times 2.0) + (-1.9731) = 1.1345$$

Fir Step 2 chalega jahan Sigmoid function naya P nikaalega:

Then Step 2 will run where the Sigmoid function will find the new P:

$$P_{\text{new}} = 1 / (1 + e^{-1.1345}) \approx 0.756$$

(Dekhiye! Hamara naya prediction 0.756 ab asaliyat 1 ke aur zyada kareeb aa gaya hai).

Fir Step 3 mein naya individual Log Loss niklega jo pehle ke 0.313 se ghatkar ab aur kam ho jayega. Phir se naye gradients niklenge aur m, c dobara isi tarah update honge.

Then in Step 3 a new individual Log Loss will be calculated which will drop even lower than the previous 0.313. New gradients will be calculated again and m , c will update in the same way.

Core Criterion: Yeh loop isi tarah non-stop tab tak ghoomta rahega, jab tak Predicted Probability (P) ekdum Actual Output (Y) ke barabar na ho jaye. Jaise hi model sikhkar $P = Y$ (yaani $1 = 1$) tak pahunchega, galti ka farq ($P - Y$) ekdum 0 ho jayega. Gradient zero hote hi m aur c update hona freeze ho jayenge aur parameters ekdum perfect location par set ho jayenge. Wahi hamara final trained model hoga jo future mein naye data par sateek prediction karega.